How does the time taken by one peer to download a file per unit of information using the BitTorrent file sharing protocol vary depending on the number of peers, their bandwidth, and the size of each file shared?

May 2026

IB Computer Science Extended Essay

[Word Count: 3991]

Table of Contents

Introduction **3**
Purpose 3
Aim of the research 3
Hypotheses 4
Background **6**
What is BitTorrent? 6
Description of the protocol 7
Ethical considerations 8
Research design **10**
Variables investigated 10
Methodology/tests performed 11
Procedure across all tests: 11
Test 1: File size 11
Test 2: Amount of peers 12
Test 3: Amount of seeds 12
Product design **13**
Network design 13
Upload/Download framework design 13
Data gathering 13
Data processing **14**
Data 14
Table 1: Download time based on file sizes 14
Table 2: Download time based on amount of downloading peers 15
Table 3: Download time based on amount of uploading seeds 15
Analysis 16
Uncertainty 16
Test 1 16
Test 2 18
Test 3 20
Evaluation **21**
Sources of error 21
Extension/improvement suggestions 21
Conclusion **23**
Bibliography **24**

Introduction

Purpose

The purpose of this investigation is to evaluate the performance of BitTorrent, a peer-to-peer (P2P) filesharing protocol, on the tasks it is commonly used for, as well as consider the viability of new applications for it. BitTorrent is currently used primarily for applications that require sharing a file consistently over a long time, such as long-term archival of data and distribution of open-source software (Hoffman, 2013). However, there are many other fields where peer-to-peer filesharing could be used, such as distributing data collected from scientific experiments or facilitating online communication. By determining the influence of variables such as amount of peers and file size on download time, this investigation will evaluate the applicability of the BitTorrent protocol to such use cases.

Aim of the research

To investigate potential applications of the BitTorrent protocol I will consider its performance relative to several parameters. These are related to the nature of the files and the network of clients, both of which might affect the download speed of BitTorrent compared to direct downloading. Since building a tracker is outside of the scope of my abilities, the investigation will focus only on parameters determined by the clients and nature of the file. As a metric of performance, I have chosen download time, measured in seconds.

First, it is interesting to consider the effect of **the number of files** on download speed. . For example, even though one file with a size of 1MB stores the same amount of data as 1024

files of size 1KB each, the latter will have vastly more metadata, which might take more time to exchange.

Next, I will consider the effect of **the amount of peers** on the time for the file to be downloaded to all peers. If only one client with a limited upload speed has the file initially, but multiple clients wish to download the file, the download speed available to the clients will be limited until more clients download the file and are able to start uploading it themselves.

Finally, it will be fruitful to consider the effect of **the amount of seeds** (clients who possess and are uploading the whole file) on download speed. Assuming a downloading client has enough download bandwidth, a file might be downloaded slower from 5 seeds with an upload speed of 4 KB/s each than from 20 seeds with an upload speed of 1 KB/s each since each additional seed might introduce overhead.

Hypotheses

Every downloaded file contains its own metadata, so I hypothesize that the total metadata of the BitTorrent download increases as more files are added. Downloading more metadata will take more time. Thus, I hypothesize that **as more files are added and total file size is kept constant, the time to download will increase.**

A somewhat accurate model is to assume that, as clients upload pieces of the file to other clients, those clients will also start uploading, so the total upload capacity of the network will increase. The total upload speed will grow **exponentially**, meaning the total time to download is **logarithmic** - each time the amount of clients is doubled, the time for all clients to download will

increase by the same amount. However, the speed-up effect that additional clients uploading will contribute is limited. The BitTorrent protocol does not coordinate between clients to create an ideal network topology, instead connecting clients based on a priority system that prioritizes each client having access to similar download speeds (Legout, Arnaud, et al, 2007). The frequent choking and unchoking that is required for this will reduce the effectiveness of additional clients uploading. The upper bound, a theoretical scenario where additional clients uploading does not speed up the downloads at all, is **linear**. Thus, I predict that **as more peers are added, the total time for the file to propagate across all peers will increase, and be described by some function with linear and logarithmic components**.

I predict that the instability of the connection and the need to perform repeated handshakes will result in a smaller percentage of available bandwidth and time being left free to download the main file compared to direct downloading, where a handshake needs only be performed once between a server and a client. Thus, **I hypothesize that with the same total bandwidth shared across peers, the time to download will increase with the number of peers.**

Background

What is BitTorrent?

BitTorrent is a peer-to-peer (**P2P**) file sharing protocol originally released by Bram Cohen in 2001. The protocol allows users to share files without a need for a central server to hold the entire file (Cohen, 2001).

Instead of holding the entire file, in BitTorrent, the central server holds a much smaller .torrent file, containing instructions how the main file is to be exchanged between users of the protocol. Additionally, it serves as a "tracker", storing the addresses of currently active peers as well as some meta information about the torrent network ("BitTorrentSpecification", 2017).

Use of BitTorrent relies on users (**clients**) who have downloaded a copy of the file, or are in the process of downloading, maintaining an Internet connection to share the file with other users. A client who is in the process of downloading a file is called a **peer**. A client who has 100% of the file and uploads it to others is called a **seed**. Peers can also upload the parts of the file that they already have, while downloading the remaining parts. A peer will maintain connections with multiple peers and seeds, downloading different parts of the file from each one and switching to others whenever a connection is interrupted ("BitTorrentSpecification", 2017).

BitTorrent is a highly viable method for file sharing, however it also faces significant drawbacks compared to direct downloading from a central server. Connections with a peer are unstable, and may be interrupted for several reasons: one of the peers might go offline, a peer might complete downloading the file pieces that the other peer/seed can provide, or a specific client might decide to switch connections for any number of internal reasons. Each time a

connection is reestablished, the peers need to perform a **handshake**, which is an overhead time and data cost (“BitTorrentSpecification”, 2017).

Description of the protocol

Even though the protocol has an official specification released by Cohen (Cohen, 2017), the accepted community standard is the unofficial *Bittorrent Protocol Specification v1.0*, which is considered more rigorously defined and up-to-date (“BitTorrentSpecification”, 2017). Thus, that is the specification I have cited and will be referring to in this investigation.

The protocol begins with the definition of a **.torrent file** - a **metadata** file that contains instructions for a client to join the network dedicated to downloading a specific file (henceforth referred to as the *main file*). A .torrent file consists of a nested data structure called a dictionary, stored as a string in a format called **bencoding** (“BitTorrentSpecification”, 2017).

Using this format, the .torrent file stores crucial metadata in one large nested dictionary. The full structure of the file will not be explained here, but the main (root) dictionary contains at least the following **required** elements:

info	A dictionary describing the file, files, or directory of files, to be downloaded
announce	The URL to the tracker server

After downloading and decoding the .torrent file, a client can send HTTP GET requests to the tracker server, requesting a list of peers. If the request is successful, the tracker will then respond with a bencoded dictionary containing a list of peers, their IP addresses, and their ports (“BitTorrentSpecification”, 2017).

The client can now attempt **handshakes** with these peers through TCP (Transmission Control Protocol). The peer and the client will verify each other using information contained in the handshake request, and a connection is established (“BitTorrentSpecification”, 2017).

A client maintains two boolean states for each connection it has - **choking** and **interested**. Peer A “choking” Peer B indicates that A will not send data packets to B. Peer A being “interested” in Peer B indicates that A will attempt to download packets from B because B has parts of the file that A is missing (“BitTorrentSpecification”, 2017).

Finally, if the peer is not choking and the client is interested, the client can send requests for data and the peer will respond with parts of the main file. This process will continue until the peer starts choking the client, the client is no longer interested, or the connection is interrupted. There is no functional distinction between a peer and a seed - once a client has downloaded the full file, it will simply become uninterested in every peer but continue sending data packets, becoming a seed (“BitTorrentSpecification”, 2017).

Ethical considerations

A potential ethical consideration of this investigation is the prevalence of BitTorrent in internet piracy. However, the protocol has a multitude of legitimate uses. BitTorrent is commonly used by FOSS (free open-source software) developers who cannot afford to host a server for direct downloads to distribute their software, for instance in the Linux OS community. It is also used to share public domain content and to archive data in a decentralized manner (Hoffman, 2013).

The experiment also uses multiple pieces of third-party software, namely the virtualization software Docker, the torrent client Transmission, and the torrent tracker

tnhnclskn/bittorrent-tracker (downloaded through Docker Desktop). All of these pieces of software are licensed under licenses that allow their use in such an investigation, which can be found in the citations (Docker Inc., 2025), (transmission, 2026), (Çalışkan, 2026).

Research design

Variables investigated

In order to test my hypotheses, I needed to isolate the three parameters I wanted to investigate, to observe their effect individually. Thus, I created the following three experiments which would test the effect of these parameters on download time.

Experiment 1: File size

Independent variable: size of each file

Dependent variable: total download time

Control variables: amount of peers, bandwidth of peers, total file size

Experiment 2: Amount of peers, same individual bandwidth

Independent variable: amount of peers

Dependent variable: total download time

Control variables: upload bandwidth of each individual peer, file size and structure

In this experiment, each new client has the same bandwidth, regardless of the amount of peers.

Experiment 3: Amount of seeds, same total bandwidth

Independent variable: amount of seeds

Dependent variable: total download time

Control variables: sum of upload bandwidth across all seeds, file size and structure

In this experiment, the same bandwidth is shared across different numbers of seeds, e.g. 1 seed with 1MB/s will be compared to 20 seeds with 50KB/s each.

Methodology/tests performed

Procedure across all tests:

1. Launch the Docker image containing the tracker and the clients.
2. Create test files of relevant sizes using the `dd` command in Terminal. The full bash script used is in the Appendix.
3. Use Transmission's MacOS desktop client to create a .torrent file for the dummy file.
4. Set the upload/download speed of the clients by editing their settings.json files.
5. Transfer the dummy file to the completed downloads folder of the seed.
6. Use a Python script that interacts with Transmission's remote control API to add the .torrent file to every relevant client. The script used is in the Appendix.
7. Wait until all files are downloaded across all relevant clients.
8. Use a Python script that interacts with Transmission's remote control API to measure the time taken to download. The script used for Test 1 as well as the script used for Tests 2 and 3 are in the Appendix.

The following are the conditions that I created for the three individual tests, designed to test each of my hypotheses.

Test 1: File size

- Amount of peers: 1
- Download bandwidth of peer capped at 50 KB/s

- Upload bandwidth of seed capped at 50 KB/s
- Seed starts with 100% of the files downloaded
- File amounts tested: 1024x1KB, 512x2KB, 256x4KB, 128x8KB, 64x16KB, 32x32KB, 16x64KB, 8x128KB, 4x256KB, 2x512KB, 1x1024KB
- Timing starts when the torrent is announced and ends when 100% has been downloaded.

Test 2: Amount of peers

- Download bandwidth of peers capped at 50 KB/s
- Upload bandwidth of seed capped at 50 KB/s
- Seed starts with 100% of the file downloaded
- File size: 1MB
- Peer amounts (excluding seed) tested: 1, 3, 5, 7, 9, 11, 13, 15, 17, 19
- Timing starts when the torrent is announced by the first client and ends when 100% has been downloaded by the last client.

Test 3: Amount of seeds

- File size: 5MB
- All clients except 1 peer start with 100% of the file downloaded
- The network upload and download speeds are capped at 1MB/s total distributed equally among the seeds, meaning each peer will have a network speed of $\frac{1MB}{peers}$. The network speeds of the downloading peer are uncapped.
- Seed amounts (excluding peer) tested: 1x1000KB/s, 10x100KB/s, 20x50KB/s
- Timing starts when the torrent is announced and ends when 100% has been downloaded.

Product design

Network design

I used the virtualization software Docker to run my tests, which creates lightweight virtual containers that run Linux applications. I created 22 Docker containers: 1 tracker running a tracker software and 21 clients running Transmission, an open source torrent client. The clients communicated with each other using a “bridge” network, a type of network that allows Docker containers to interact with networks on the computer by assigning ports to them and treating them as regular processes hosted on the computer.

Upload/Download framework design

In order to ensure that I could start uploads and downloads at the same time across clients, I wrote a Python script that adds the torrent to all of the client containers at the same time. This was necessary to eliminate human error that would affect the recorded times if I were to start the downloads manually.

Data gathering

Data was collected from the clients using a Python script I wrote implementing Transmission’s Remote Control API (transmission, 2025), which allows me to access logs with timestamps indicating when a .torrent file was announced and when it finished downloading.

Data processing

Data

The following is the data I obtained from my experiments, listing the download times under the outlined conditions. I repeated each test several times in order to reduce random error and calculate an average time.

Table 1: Download time based on file sizes

1: File Size							
File amount x size	**Trial 1(s)**	**Trial 2(s)**	**Trial 3(s)**	**Trial 4(s)**	**Trial 5(s)**	**Average(s)**	**Uncertainty(s)**
1024x1KB	32	32	32	32	32	32	1
512x2KB	32	32	31	31	32	31.6	1.5
256x4KB	32	32	32	32	32	32	1
128x8KB	32	32	32	32	32	32	1
64x16KB	32	32	32	31	33	32	2
32x32KB	31	32	31	32	32	31.6	1.5
16x64KB	32	32	32	32	31	31.8	1.5
8x128KB	31	32	31	31	32	31.4	1.5
4x256KB	32	32	32	32	35	32.6	2.5
2x512KB	32	32	32	32	32	32	1
1x1024KB	32	31	32	32	32	31.8	1.5

Figure 1: The table with the results from the first experiment. Each row contains the obtained download time values for a different number of files.

Table 2: Download time based on amount of downloading peers

2: Peer Amount, Same Individual Bandwidth (50KB/s)					
Amount of peers (-seed)	**Trial 1(s)**	**Trial 2(s)**	**Trial 3(s)**	**Average**	**Uncertainty**
1	32	32	32	32.0	1
3	76	57	61	64.7	10.5
5	87	82	89	86.0	4.5
7	103	110	92	101.7	10
9	122	121	116	119.7	4
11	128	143	132	134.3	8.5
13	176	146	153	158.3	16
15	175	189	173	179.0	9
17	204	198	195	199.0	5.5
19	227	207	212	215.3	11

Figure 2: The table with results from the second experiment. Each row contains the obtained download time values for a different number of peers.

Table 3: Download time based on amount of uploading seeds

3: Peer Amount, Same Total Bandwidth (1 MB/s)			
Peers x Bandwidth per peer	**1x1000KB/s**	**10x100KB/s**	**20x50KB/s**
Trial 1	22	24	24
Trial 2	21	23	23
Trial 3	21	23	23
Trial 4	23	24	23
Trial 5	22	23	24
Trial 6	22	23	24
Trial 7	22	23	24
Trial 8	21	23	25
Trial 9	23	23	24
Trial 10	21	23	23
Average	21.8	23.2	23.7
Uncertainty	2	1.5	2

Figure 2: The table with results from the third experiment. Each column contains the obtained download time values for a different number of seeds, sharing the same total upload bandwidth.

Analysis

Uncertainty

The uncertainty for the experiment comes from two sources: the differences between the trials and the uncertainty of the time measurement provided by Transmission's logging. Time was measured by subtracting the end time of a torrent download from the start time. Both have an uncertainty of ±0.5s for a total of ±1s. For the difference between trials, the formula

$$\text{Uncertainty} = \frac{max - min}{2}$$

was used. Total uncertainty was calculated by summing the uncertainty from the trials and from the time measurement.

Test 1

As seen in Table 1, the average time to download did not change significantly as the number of files increased, being equal to **32s** rounded to the same number of significant figures as the data across almost all file sizes with minimal variation. Thus, it can be concluded that **the time to download is not significantly affected by the number of files to be downloaded.**

This can be explained by the efficiency of the BitTorrent algorithm in handling multi-file structures. Instead of downloading metadata describing the separate files from clients, a .torrent file directly encodes the file structure.

We can thus expect the size of the .torrent file to grow significantly as the number of files increases. If the number of files becomes too large, the .torrent file itself, which has to be distributed before the BitTorrent download can begin, might become too large and significantly complicate distribution.

The file size of the torrents used confirms this idea, as for larger file amounts the size of the .torrent file starts growing rapidly at a linear rate to the number of files.

.torrent size	
File amount	**Size(KB)**
1024x1KB	1
512x2KB	1
256x4KB	1
128x8KB	1
64x16KB	2
32x32KB	2
16x64KB	4
8x128KB	6
4x256KB	11
2x512KB	21
1x1024KB	41

Figure 4: The table showing the relationship between the size of the .torrent file and the number of files in the directory that the .torrent file describes.

This is not an issue for many practical applications of filesharing. However, some, such as system logs or archives from software, certain software repositories, or large experiment datasets, might contain extremely large amounts of small files. Thus, it can be concluded that the base BitTorrent protocol will be less effective at sharing these types of files, since a large .torrent file must be distributed before P2P file sharing can occur.

We can also use the results from this test to determine the overhead added by the BitTorrent clients to the time required to download a file. A file of size 1MB at a speed of 50KB/s should normally take $\frac{1024}{50} = 20.48$ s to download, however the average time for the 1x1024MB file was 31.8s. Thus, the overhead taken by the client to establish a connection is $31.8 - 20.48 = 11.32 \approx 11s \pm 1.5s$. An overhead of **11s** means that the BitTorrent protocol, although highly effective for sharing large files, is extremely impractical for applications that require fast back-and-forth sharing of files, such as online communication.

Test 2

The BitTorrent algorithm establishes a priority system for peers. A client only uploads to (unchokes) a maximum of 4 peers due to TCP becoming unstable over more concurrent uploads. The peers are prioritized in a way that attempts to ensure similar download speeds for each peer, as long as they contribute to uploading. This will result in the file being spread around evenly and downloaded at the same time on all clients (Legout, Arnaud, et al., 2007).

If clients coordinated perfectly, then upload speed would increase proportionally to the propagation of the file, so total download time would grow as a **logarithmic** function. However, a client only checks whether to reassign priority and unchoke new clients at set intervals. Thus, every new set of clients will need to wait until previous clients that now have new pieces of the file reassign their priorities and begin uploading to them (Legout, Arnaud, et al.). Each additional peer introduced will increase the amount of time taken to choke/unchoke, and this additional time will grow as a **linear** function of the number of peers.

Since the size of the test file used was the same for this test as the 1MB file in Test 1 and all other variables are accounted for, we can conclude that the overhead for establishing an initial connection between clients will be the same in this test. Since the data for n=1 client is extremely reliable with little to no uncertainty across both tests, we can use it as a baseline to determine the proportionality factor, which is equal to $total - overhead = 32 - 11 = 21$.

Hence, we can model the data as an interpolation of linear factor $g(n) = 21n$ and logarithmic factor $h(n) = 21log_2 n$,[1] where n is the number of peers, expressed as

$$f(n) = b * g(n) + (1 - b) * h(n) + 11 \text{ where } 0 < b < 1.$$

[1] For discrete logarithmic processes (i.e. an individual client either is uploading or isn't, without the possibility of fractions), log2 is used rather than ln.

Here, b is the interpolation factor, a variable we introduce to control how much each term contributes to the composite function. The smaller b is, the more the logarithmic component contributes to the function's growth, meaning the speed-up provided by additional clients is more significant. Fitting the b to the experimental dataset seen in Table 2 using a Python script yields:

b=0.3592, **RMSE = 3.6577, RMSPE = 3.2863%**.

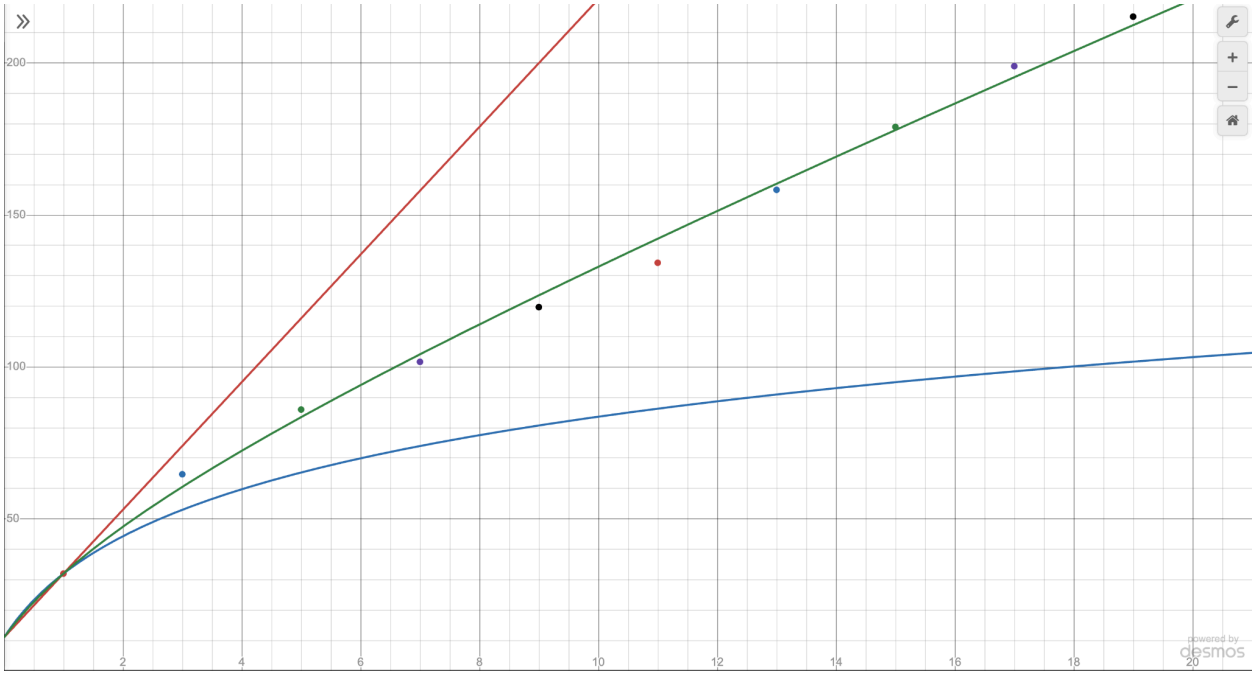


Figure 5: Graph showing the data points, the linear component (red), logarithmic component (blue), and composite function (green). Created in Desmos.

RMSE (root mean square error) is a measure of the average absolute deviation from the data points, and RMSPE (root mean square percentage error) is the average percentage deviation from the data points (ScienceDirect, 2022). The relatively low RMSPE suggests that the hypothesis is correct: **as the number of peers increases, the time for the file to propagate across all peers increases as a combination of linear and logarithmic components.** To be more precise, the linear component contributes $\frac{0.3592}{1.0} * 100 = \mathbf{36\%}$. Since the linear component will always dominate the logarithmic component as n increases, we can conclude that

for the given bandwidth and file size, as the amount of peers increases, the time to download across all peers will approach 36% of the time required for direct downloading. This finding, however, cannot be generalized to all file sizes and bandwidths.

The presence of a significant linear component in the download time means that the initial propagation of a file across new peers is far from optimal. While this is a less significant issue for use cases where a .torrent file is shared over a long period of time, it implies that the algorithm would show suboptimal performance in applications where a file only has to be shared once, or new files have to be shared regularly.

Test 3

This test was designed to measure how effectively a peer is able to download from multiple seeds without losing bandwidth from having to perform multiple handshakes with separate clients[2].

The data reveals that the time to download a file from several peers **does increase, but very slightly**. Due to the variation being extremely small, I am unable to confidently describe the experimental data with a function. However, my findings highlight a key strength of the BitTorrent protocol: **once enough clients download the file and become seeds, the time taken for a new client to download the file is almost the same as in a direct download with the same total bandwidth.** This means that the algorithm is very effective for long-standing .torrent files with a large amount of seeds, such as open-source software that continues to be used over a long time. To directly answer the hypothesis, **the time taken by a client to download a file increases very slightly as the amount of seeds increases, keeping total bandwidth constant.**

[2] The downloading client's download speed was unlimited, so the loss would come from the limited upload speeds of the uploading seeds being occupied by handshake packets.

Evaluation

Sources of error

The main source of error for this experiment was **random error** in Transmission's log data. Since the API only logs timestamps to the nearest second, a relatively large amount of uncertainty was introduced.

Another potential source of error could be the inconsistency of the network over which files are downloaded. However, I was able to largely mitigate this error by using Docker's "bridge" network mode within my computer, which has consistently low latency since all communication happens on the hardware of my computer.

Finally, a poorly functioning BitTorrent tracker could introduce **systemic error** by incorrectly reporting when a torrent file was announced back to the client, which would lead to the client incorrectly logging the time. However, any error introduced in this manner was accounted for in the uncertainty of the overhead calculation.

Extension/improvement suggestions

A major improvement that could be introduced is the use of a .torrent client with more precise logging and the use of larger .torrent files leading to longer download times. Both options would reduce the percentage uncertainty of the time measurements, improving the accuracy of the experiment.

Since Test 2 yielded the most meaningful results, an extension opportunity is to attempt to generalize its results to different file sizes or network speeds. By exploring how the

contribution of the linear component to the growth function changes, we could determine whether or not the protocol becomes more efficient for larger files that take longer to download.

Conclusion

This investigation examined the effectiveness of BitTorrent, a P2P filesharing protocol, in reducing download times for files by allowing independent clients to download pieces of the files from each other. It found that download times are not influenced by the structure of the directories downloaded, however a workaround such as archiving must be used to share directories with extremely large amounts of files. In Test 2, it further established that the algorithm has a suboptimal performance in situations where a file must propagate from one seed to many peers, however in Test 3 it found that the algorithm performs extremely well in situations where a client can download a file from many seeds simultaneously. These facts suggest that the protocol is better suited to situations where a file must be shared many times over a long period of time rather than situations where a new file must be shared once. Finally, it established that the protocol requires a significant amount of overhead time for clients to establish a connection, suggesting that it is not suitable for use cases involving back-and-forth communication.

Overall, the investigation went according to plan. The experiment proceeded without unexplainable errors. One of my hypotheses, the effect of the number of files, was disproven, however I was able to find a reason for these results. The hypothesis regarding the amount of peers was only proven for a specific configuration, and further investigation is necessary to determine how download times change for different file sizes and different network topologies.

Bibliography

"BitTorrentSpecification" *Theory.org*, 2017, wiki.theory.org/BitTorrentSpecification#Identification. Accessed 10 May 2025.

Cohen, Bram. "Bep_0003.Rst_post." *Bittorrent.org*, 2017, bittorrent.org/beps/bep_0003.html. Accessed 7 Sept. 2025.

Cohen, Bram. "Decentralization : Message: BitTorrent - a New P2P App." *Yahoo Finance*, 2001, web.archive.org/web/20080129085545/finance.groups.yahoo.com/group/decentralization/message/3160. Accessed via *archive.org*, 8 Feb. 2026.

Docker Inc. "Docker Desktop License Agreement." *Docker Documentation*, Docker Inc, 2025, docs.docker.com/subscription/desktop-license. Accessed 8 Feb. 2026.

EBSCO. "BitTorrent | Architecture | Research Starters | EBSCO Research." *EBSCO*, 2024, www.ebsco.com/research-starters/architecture/bittorrent. Accessed 8 Feb. 2026.

Hoffman, Chris. "8 Legal Uses for BitTorrent: You'd Be Surprised." *MUO*, 17 Aug. 2013, www.makeuseof.com/tag/8-legal-uses-for-bittorrent-youd-be-surprised/. Accessed 7 Sept. 2025.

kenorb. "GitHub - Kenorb-Contrib/BitTorrent: Python-Based Reference Implementation of BitTorrent Client." *GitHub*, 2015, github.com/kenorb-contrib/BitTorrent. Accessed 7 Sept. 2025.

Legout, Arnaud, et al. "Clustering and Sharing Incentives in BitTorrent Systems." *ACM SIGMETRICS Performance Evaluation Review*, vol. 35, no. 1, 12 June 2007, pp. 301–312, https://doi.org/10.1145/1269899.1254919. Accessed 24 Apr. 2021.

"Pythorrent." *PyPI*, 6 Apr. 2017, pypi.org/project/pythorrent/. Accessed 5 Sept. 2025.

ScienceDirect. “Root Mean Square Error - an Overview | ScienceDirect Topics.” *Www.sciencedirect.com*, 2022, www.sciencedirect.com/topics/engineering/root-mean-square-error. Accessed 5 Feb. 2026.

transmission. “GitHub - Transmission/Transmission: Official Transmission BitTorrent Client Repository.” *GitHub*, 27 Jan. 2026, github.com/transmission/transmission. Accessed 4 Feb. 2026.

transmission. “Transmission’s RPC Specification.” *GitHub*, 2025, github.com/transmission/transmission/blob/main/docs/rpc-spec.md. Accessed 5 Feb. 2026.

Widman, Kristen. “How to Write a Bittorrent Client, Part 1 | Kristenwidman.” *Kristenwidman.com*, 23 Nov. 2012, www.kristenwidman.com/blog/33/how-to-write-a-bittorrent-client-part-1/. Accessed 7 Oct. 2025.

Çalışkan, Tunahan. “Tnhnclskn/Bittorrent-Tracker - Docker Image.” *Docker.com*, 2026, hub.docker.com/r/tnhnclskn/bittorrent-tracker. Accessed 2 Feb. 2026.

Appendix A: bash scripts used for file generation

The following scripts were used to generate test files of the required sizes. They were run in my MacOS computer's Terminal, and are written in the bash script language which Terminal uses.

Test 1

```
mkdir testfolder_1024x1
for i in {1..1024}; do
 dd if=/dev/zero of=testfolder_1024x1/file_$i.bin bs=1k count=1
done

mkdir testfolder_512x2
for i in {1..512}; do
 dd if=/dev/zero of=testfolder_512x2/file_$i.bin bs=2k count=1
done

mkdir testfolder_256x4
for i in {1..256}; do
 dd if=/dev/zero of=testfolder_256x4/file_$i.bin bs=4k count=1
done

mkdir testfolder_128x8
for i in {1..128}; do
 dd if=/dev/zero of=testfolder_128x8/file_$i.bin bs=8k count=1
done

mkdir testfolder_64x16
for i in {1..64}; do
 dd if=/dev/zero of=testfolder_64x16/file_$i.bin bs=16k count=1
done

mkdir testfolder_32x32
for i in {1..32}; do
 dd if=/dev/zero of=testfolder_32x32/file_$i.bin bs=32k count=1
done
```

```
mkdir testfolder_16x64
for i in {1..16}; do
 dd if=/dev/zero of=testfolder_16x64/file_$i.bin bs=64k count=1
done

mkdir testfolder_8x128
for i in {1..8}; do
 dd if=/dev/zero of=testfolder_8x128/file_$i.bin bs=128k count=1
done

mkdir testfolder_4x256
for i in {1..4}; do
 dd if=/dev/zero of=testfolder_4x256/file_$i.bin bs=256k count=1
done

mkdir testfolder_2x512
for i in {1..2}; do
 dd if=/dev/zero of=testfolder_2x512/file_$i.bin bs=512k count=1
done

mkdir testfolder_1x1024
for i in {1..1}; do
 dd if=/dev/zero of=testfolder_1x1024/file_$i.bin bs=1024k count=1
done
```

Test 2

```
dd if=/dev/zero of=./file_1mb.bin bs=1M count=1
```

Test 3

```
dd if=/dev/zero of=./file_10mb.bin bs=1M count=10
```

Appendix B: Python script to add a torrent file to multiple Transmission clients

This Python script uses “requests”, a library that can send requests to servers, to communicate with my Transmission clients, which are hosted locally on my computer. It then implements the Transmission remote control API to communicate with all my clients simultaneously and add the same torrent file to all of them, so they can start downloading. This is done to remove delay from manually adding torrent downloads to each client and ensuring consistent timing.

```
#Script that adds .torrent files to every client.
import requests
import base64
clients = [
    "http://localhost:9092/transmission/rpc",
    "http://localhost:9093/transmission/rpc",
    #...full list of clients here
]
#Specify name of file here
torrent_file = "files and torrents/file_5mb.torrent"
with open(torrent_file, "rb") as f:
    torrent_data = base64.b64encode(f.read()).decode()
for url in clients:
    #get session id
    r = requests.post(url)
    session_id = r.headers.get("X-Transmission-Session-Id")
    payload = {
        "method": "torrent-add",
        "arguments": {
            "metainfo": torrent_data
        }
    }
    headers = {"X-Transmission-Session-Id": session_id}
    r = requests.post(url, json=payload, headers=headers)
    print(f"{url}: {r.json()}")
```

Appendix C: Python script to see how long downloading took on a given client (used in Tests 1 and 3)

This Python script uses “requests”, a library that can send requests to servers, to communicate with my Transmission clients, which are hosted locally on my computer. It then implements the Transmission remote control API to get timing data from a client. Clients record when a download starts, how much is downloaded, and when a download is completed, which allows this script to either display a percentage of the file downloaded or display how long it took to download if the download is complete.

```
#Script to find time taken for Test 1.
import requests
import datetime
clients = [
    #Seed
    "http://localhost:9092/transmission/rpc",
    #Peer
    "http://localhost:9093/transmission/rpc",
]
for url in clients:
    #Get session ID
    r = requests.post(url)
    session_id = r.headers.get("X-Transmission-Session-Id")
    headers = {"X-Transmission-Session-Id": session_id}
    #Specify what to request from API
    payload = {
        "method": "torrent-get",
        "arguments": {
            "fields": ["name", "startDate", "doneDate", "percentDone", "isFinished"]
        }
    }
    r = requests.post(url, json=payload, headers=headers)
    data = r.json()["arguments"]["torrents"]
    print(f"\n{url}")
    for t in data:
        if t["percentDone"] >= 1.0:
            start = datetime.datetime.fromtimestamp(t["startDate"])
            done = datetime.datetime.fromtimestamp(t["doneDate"])
            duration = done - start
            print(f"{t['name']}: finished, took {duration} from {start} to {done}")
        else:
            print(f"{t['name']}: {t['percentDone']*100:.1f}% done")
```

Appendix D: Python script to see how long it takes all clients to download the file (used in Test 2)

This Python script uses “requests”, a library that can send requests to servers, to communicate with my Transmission clients, which are hosted locally on my computer. It then implements the Transmission remote control API to get timing data from a client. Clients record when a download starts, how much is downloaded, and when a download is completed.
This script collects this data from all the clients involved in the experiment, and displays the total time it took for all clients to download a file. It does this by finding the difference between when the first download starts and when the last download ends.

```
#Script to find time taken for Tests 2 and 3.
import requests
import datetime
from collections import defaultdict
clients = [
    #Peers only, not including seed
    "http://localhost:9093/transmission/rpc",
    "http://localhost:9094/transmission/rpc",
    "http://localhost:9095/transmission/rpc",
    #full list of clients here
]
# Store start and done times
torrent_times = defaultdict(lambda: {"starts": [], "dones": []})
for url in clients:
    r = requests.post(url)
    session_id = r.headers.get("X-Transmission-Session-Id")
    headers = {"X-Transmission-Session-Id": session_id}
    payload = {
        "method": "torrent-get",
        "arguments": {
            "fields": ["name", "startDate", "doneDate", "percentDone", "isFinished"]
        }
    }
    r = requests.post(url, json=payload, headers=headers)
    torrents = r.json()["arguments"]["torrents"]
    for t in torrents:
        if t["percentDone"] >= 1.0:
            if t["startDate"] and t["startDate"] > 0:
                torrent_times[t["name"]]["starts"].append((t["startDate"], url))
            if t["doneDate"] and t["doneDate"] > 0:
                torrent_times[t["name"]]["dones"].append((t["doneDate"], url))
```

```
print("\nTest results:")
for name, times in torrent_times.items():
    first_start_time, first_client = min(times["starts"], key=lambda x: x[0])
    last_done_time, last_client = max(times["dones"], key=lambda x: x[0])
    first_start_dt = datetime.datetime.fromtimestamp(first_start_time)
    last_done_dt = datetime.datetime.fromtimestamp(last_done_time)
    swarm_duration = last_done_dt - first_start_dt
    print(f"{name}:")
    print(f"  First start: {first_start_dt} on {first_client}")
    print(f"  Last done : {last_done_dt} on {last_client}")
    print(f"  Swarm duration: {swarm_duration}")
```

Appendix E: Docker Compose file used to create the containers

This is the .yaml configuration file used to create new containers running the required clients in Docker Compose. I used this to create 1 container running a tracker, and 21 containers running Transmission clients (the full code is shortened here, only showing the configuration for 1 of the Transmission clients, but the rest are identical except for their port numbers and IDs). I used this configuration file to set up the bridge network between them, allowing them to communicate directly on my computer hardware.

```
version: "3.6"
networks:
 torrent-network-1:
   driver: bridge
   driver_opts:
     parent: eth0
   ipam:
     config:
       - subnet: 192.168.1.0/24
         gateway: 192.168.1.1
services:
 #Torrent tracker
 tracker:
   image: tnhnclskn/bittorrent-tracker
   container_name: tracker
   ports:
     - "6969:6969"
   networks:
     - torrent-network-1
   restart: unless-stopped
 #One example of a container. A total of 21 of these were made, numbered sequentially.
 tm-container-01:
   container_name: "tm-container-01"
   entrypoint:
     - "/init"
   environment:
     - "PATH=/lsiopy/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
     - "PS1=$$(whoami)@$$(hostname):$$(pwd)\\$$ "
     - "HOME=/root"
     - "TERM=xterm"
     - "S6_CMD_WAIT_FOR_SERVICES_MAXTIME=0"
     - "S6_VERBOSITY=1"
```

```
      - "S6_STAGE2_HOOK=/docker-mods"
      - "VIRTUAL_ENV=/lsiopy"
      - "LSIO_FIRST_PARTY=true"
    hostname: "a55685b89173-1"
    image: "linuxserver/transmission:latest"
    ipc: "private"
    labels:
      build_version: "Linuxserver.io version:- 4.0.6-r4-ls311 Build-date:-
2025-09-23T10:39:34+00:00"
      maintainer: "aptalca"
      org.opencontainers.image.authors: "linuxserver.io"
      org.opencontainers.image.created: "2025-09-23T10:39:34+00:00"
      org.opencontainers.image.description:
"[Transmission](https://www.transmissionbt.com/) is designed\
       \ for easy, powerful use. Transmission has the features you want from a
BitTorrent client: encryption,\
       \ a web interface, peer exchange, magnet links, DHT, µTP, UPnP and NAT-PMP port
forwarding, webseed\
       \ support, watch directories, tracker editing, global and per-torrent speed
limits, and more."
      org.opencontainers.image.documentation:
"https://docs.linuxserver.io/images/docker-transmission"
      org.opencontainers.image.licenses: "GPL-3.0-only"
      org.opencontainers.image.ref.name: "df00bc19950c10334e194b6eb8bf6b3514030a5e"
      org.opencontainers.image.revision: "df00bc19950c10334e194b6eb8bf6b3514030a5e"
      org.opencontainers.image.source:
"https://github.com/linuxserver/docker-transmission"
      org.opencontainers.image.title: "Transmission"
      org.opencontainers.image.url:
"https://github.com/linuxserver/docker-transmission/packages"
      org.opencontainers.image.vendor: "linuxserver.io"
      org.opencontainers.image.version: "4.0.6-r4-ls311"
    logging:
      driver: "json-file"
      options: {}
    networks:
      - torrent-network-1
    ports:
      - "0:51414/tcp"
      - "0:51414/udp"
      - "9092:9091/tcp"
    volumes:
```

```
      - "/[DIRECTORY]/config:/config"
      - "/[DIRECTORY]/t1/downloads:/downloads"
      - "/[DIRECTORY]/t1/watch:/watch"
    working_dir: "/"
```

Appendix F: Python script to fit function to test data

This Python script that I created for Test 2 uses scipy, a data analysis library, to fit a parameter in my function to the data points I obtained from my test. It adjusts the parameter until a value is found that matches the data points as closely as possible, outputting the optimal value of the parameter. In this case, the parameter was the interpolation factor b, which indicates how much the growth rate is dependent on a linear factor and how much it is dependent on a logarithmic factor. It also calculated the RMSE (root mean square error), which is a measure of the average deviation of my data points from the function. A small deviation means my hypothesis is likely correct.

```
import numpy as np
from scipy.optimize import curve_fit


def lin(x): return 21*x + 11
def log(x): return 21*np.log2(x+1) + 11
def composite(x, b): return b*lin(x) + (1-b)*log(x)
x_data = np.array([1,3,5,7,9,11,13,15,17,19])
y_data = np.array([32.0,64.7,86.0,101.7,119.7,134.3,158.3,179.0,199.0,215.3])
popt, pcov = curve_fit(composite, x_data, y_data, p0=[0.5])
b = popt[0]
print(f"Best-fit b: {b}")

y_predicted = composite(x_data, popt)
rmse = np.sqrt(np.mean((y_data - y_predicted)**2))
print(f"RMSE: {rmse}")
```